Riya Varyani D20A 61

**Experiment No.1**

**Aim:** Cryptography in Blockchain, Merkle root Tree Hash

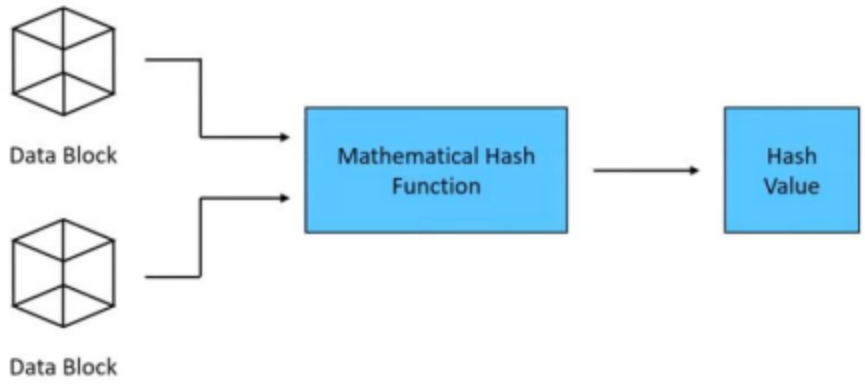
**Tasks Performed:**

1. **Hash Generation using SHA-256:**
   - Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
   - Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
   - Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
   - Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

# Cryptographic Hash Functions in Blockchain

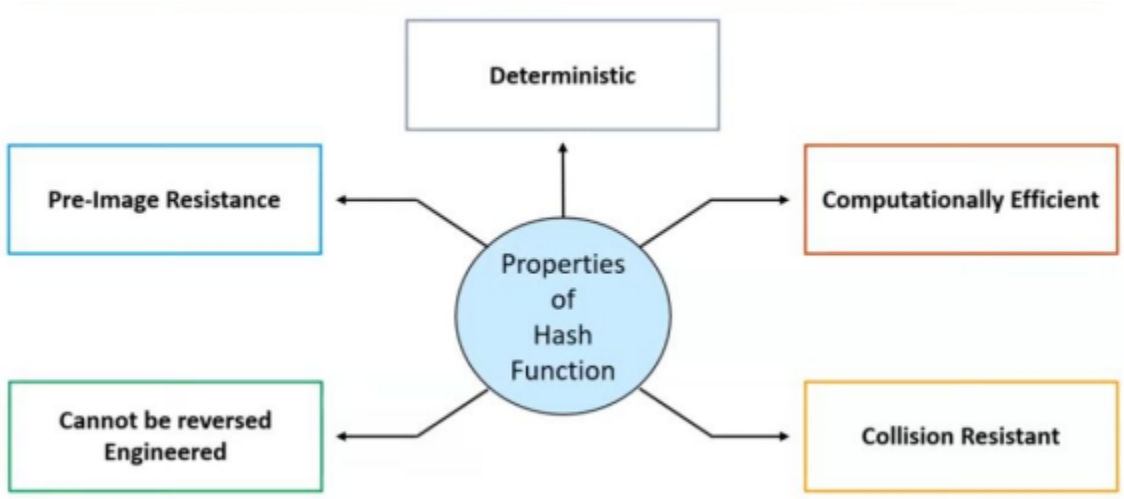
Cryptographic hash functions are deterministic mathematical algorithms that transform an input of arbitrary size into a fixed-length string of characters, known as a hash or message digest. The output appears random and is unique to the specific input; even a minute change in the input produces a vastly different hash. In blockchain technology, hash functions are fundamental to ensuring data integrity and security. A widely used hash function in many blockchain systems is SHA-256 (Secure Hash Algorithm 256-bit), which outputs a 256-bit (64 hexadecimal characters) hash.

Riya Varyani D20A 61




Key properties of cryptographic hash functions are:

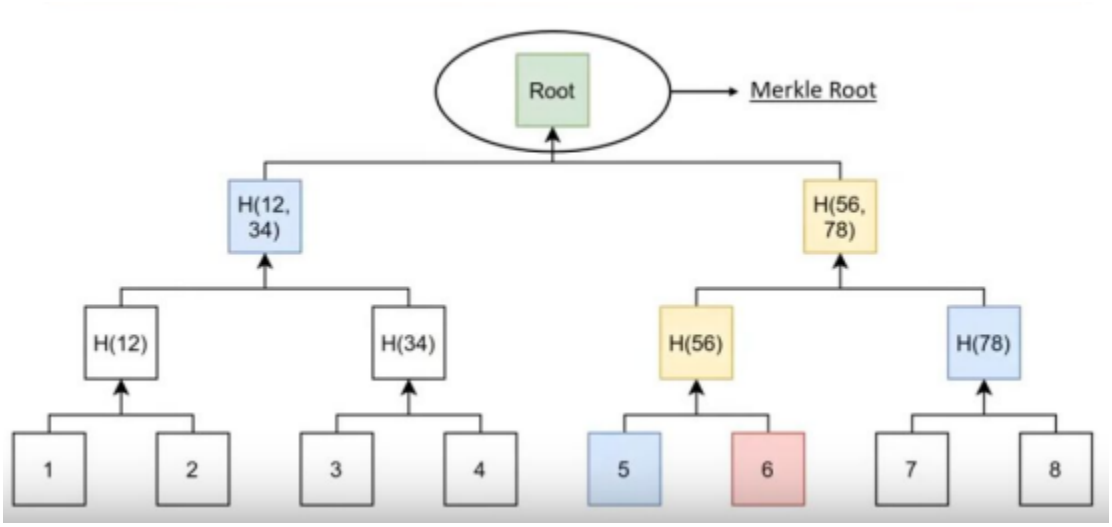
- **Determinism:** The same input always produces the same output.
- **Fixed Output Length:** Regardless of input size, the hash length remains constant.
- **Pre-image Resistance:** It is computationally infeasible to reconstruct the original input from its hash.
- **Collision Resistance:** It is extremely unlikely for two different inputs to produce the same hash.
- **Avalanche Effect:** Even a small alteration in the input produces a completely different hash output.



In blockchain, these properties protect the immutability of blocks. Each block contains the hash of the previous block; any tampering with a block would change its hash, invalidating the entire chain.

## What is a Merkle Tree?

A Merkle Tree, named after Ralph Merkle, is a binary tree data structure in which every non-leaf node is the cryptographic hash of its two child nodes. The leaf nodes consist of the hashes of individual data pieces, such as transactions. Above these, pairs of leaf hashes are concatenated and hashed again to form parent hashes. This process continues iteratively until only one hash remains at the top of the tree, called the **Merkle Root**.




A Merkle Tree provides an efficient and secure means of summarizing and verifying the integrity of large sets of data. It is widely adopted in distributed systems and blockchain networks where it enables light clients to verify transactions without downloading complete datasets.

## What is a Cryptographic Puzzle and the Golden Nonce?

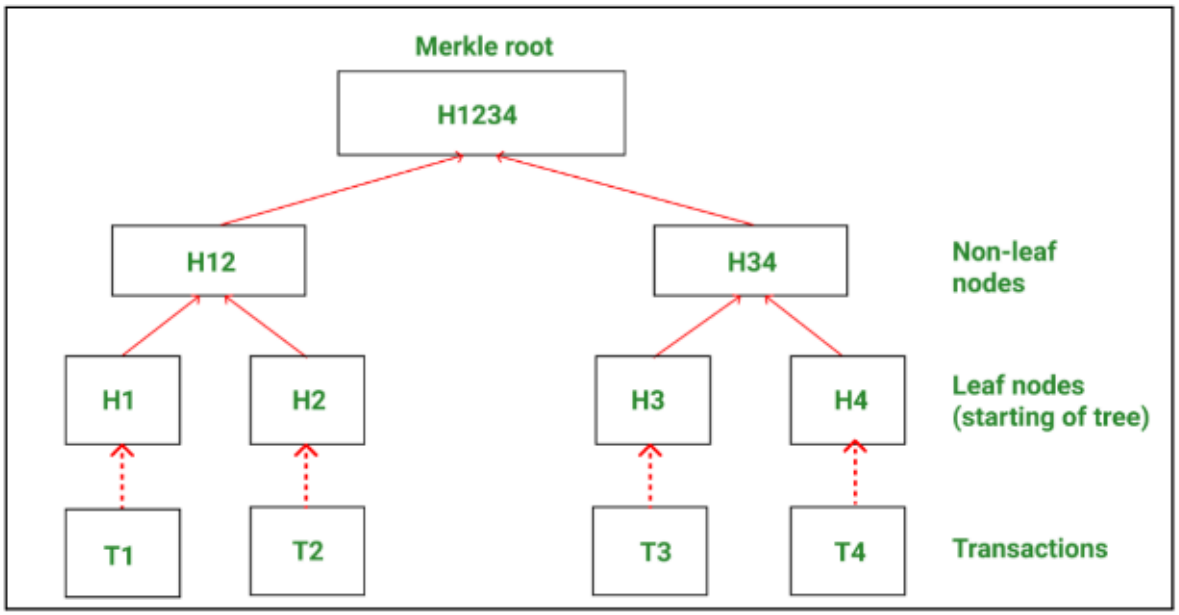
A **cryptographic puzzle** refers to a computational challenge that requires significant processing effort to solve but is easy to verify once found. In blockchain consensus

mechanisms like Proof of Work (PoW), miners compete to solve a cryptographic puzzle by finding a specific value called a **nonce** (number used once). The puzzle usually involves finding a nonce that, when combined with block data and passed through a hash function (e.g., SHA-256), yields a hash that satisfies a predetermined condition, such as having a specific number of leading zeros.

The **Golden Nonce** is the particular nonce value that satisfies the difficulty condition of the cryptographic puzzle. Once a miner finds this value, the resulting hash meets the network's target requirement and the block is considered valid. The discovery of the Golden Nonce demonstrates that the miner has expended computational work, justifying the reward issued by the network.

## How Does a Merkle Tree Work?

1. **Hash Individual Transactions:** Each transaction in a dataset is hashed using a cryptographic hash function.
2. **Pair and Hash:** The hashes of individual transactions are paired and concatenated. Each pair is then hashed to produce a parent hash.
3. **Handle Odd Numbers:** If there is an odd number of hashes at any level, the last hash is duplicated to form a pair.
4. **Iterate Upward:** The process of pairing and hashing continues iteratively up the tree structure until only one hash remains.
5. **Merkle Root:** The final single hash at the top of the tree is the Merkle Root. It represents a compact summary of all underlying transactions.




The Merkle Root is stored in the block header and serves as a fingerprint for the entire set of transactions in that block.

## Benefits of Merkle Tree

Merkle Trees offer several advantages in data processing and blockchain systems:

- **Efficient Verification:** Allows verification of individual data elements without accessing the entire dataset.
- **Reduced Storage Needs:** Only the Merkle Root and relevant branch data are needed for proof of inclusion.
- **Fast Integrity Checking:** Any modification to underlying data changes the corresponding leaf hash and propagates upward, making tampering evident.
- **Tamper Detection:** Alterations in any transaction lead to mismatches in intermediate hashes and ultimately the Merkle Root.
- **Scalability:** Suitable for large datasets as the structure reduces the computational and storage overhead required for verification.

Riya Varyani D20A 61

# Use Cases of Merkle Tree

Merkle Trees are employed in various areas of computer science and distributed systems:

- **Blockchain Technology:** Used to summarize and verify transactions in blocks (e.g., Bitcoin, Ethereum).
- **Distributed Databases:** Enable efficient integrity checks across replicated data.
- **Peer-to-Peer Networks:** Facilitate partial verification of files (e.g., BitTorrent).
- **Version Control Systems:** Used in systems like Git to detect changes and manage versions.
- **Cloud Storage Services:** Assist in verifying integrity of stored and retrieved data without exposing full content.

**Programs and Output:**

### Program #1: Python program that uses the hashlib library to create the hash of a given string

```
import hashlib   # correct import


def create_hash(string):

    # Create a hash object using SHA-256 algorithm

    hash_object = hashlib.sha256()


    # Convert the string to bytes and update the hash object

    hash_object.update(string.encode('utf-8'))


    # Get the hexadecimal representation of the hash

    hash_string = hash_object.hexdigest()
```

```
    # Return the hash string

    return hash_string

# Example usage

input_string = input("Enter a string: ")

hash_result = create_hash(input_string)

print("Hash:", hash_result)
```

```
Enter a string: Riya
Hash: beb0256ed307472fba4f4ff3e82c68d84772540db7c9a816275fc774c835a75e
```

**Program #2: Program to generate required target hash with input string and nonce**

```
import hashlib

# Get user input

input_string = input("Enter a string: ")

nonce = input("Enter the nonce: ")

# Concatenate the string and nonce

hash_string = input_string + nonce

# Calculate the hash using SHA-256

hash_object = hashlib.sha256(hash_string.encode('utf-8'))

hash_code = hash_object.hexdigest()



# Print the hash code

print("Hash Code:", hash_code)
```

Riya Varyani D20A 61

```
Enter a string: Riya
Enter the nonce: 4
Hash Code: 70511d27149f98381bd54eff5a9052b5ce5ed45f2dadc846d20de10f5917a3c7
```

**Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string**

```python
import hashlib

def find_nonce(input_string, num_zeros):

    nonce = 0

    hash_prefix = '0' * num_zeros

    while True:

        hash_string = input_string + str(nonce)

        hash_object = hashlib.sha256(hash_string.encode('utf-8'))

        hash_code = hash_object.hexdigest()

        if hash_code.startswith(hash_prefix):

            print("Hash:", hash_code)

            return nonce

        nonce += 1

input_string = input("Enter the input string: ")

num_zeros = int(input("Enter number of leading zeros: "))

expected_nonce = find_nonce(input_string, num_zeros)

print("\nInput String:", input_string)

print("Leading Zeros:", num_zeros)

print("Expected Nonce:", expected_nonce)
```

Riya Varyani D20A 61

```
Enter the input string: Riya
Enter number of leading zeros: 3
Hash: 000fd334835536108980d8b17569b5e2dadc8603802ca8f29930cf6be3ac9565

Input String: Riya
Leading Zeros: 3
Expected Nonce: 967
```

**Program 4: Generating Merkle Tree for given set of Transactions**

```
import hashlib

def build_merkle_tree(transactions):

    if len(transactions) == 0:

        return None

    if len(transactions) == 1:

        return transactions[0]

    # Recursive construction of the Merkle Tree

    while len(transactions) > 1:

        # If odd number of transactions, duplicate last one

        if len(transactions) % 2 != 0:

            transactions.append(transactions[-1])

        new_transactions = []

        for i in range(0, len(transactions), 2):

            combined = transactions[i] + transactions[i+1]

                                                        hash_combined     =
hashlib.sha256(combined.encode('utf-8')).hexdigest()

            new_transactions.append(hash_combined)
```

```
        transactions = new_transactions

    return transactions[0]

# Example usage

transactions = [

    "Transaction 1",

    "Transaction 2",

    "Transaction 3",

    "Transaction 4",

    "Transaction 5"

]

merkle_root = build_merkle_tree(transactions)

print("Merkle Root:", merkle_root)
```

```
Merkle Root: a4a18941de1162b17a46c4f8c87d8a0850b46fad17ac881340061d9233785077
```

**Conclusion:**

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).

- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.